Laboratorio 1

Walter Alfaro Ulate, Allan Arrieta, Santos Alexander Lopez Valles, Esteban Vargas

***Resumen*—En este laboratorio se estudió el flujo básico de diseño digital utilizando SystemVerilog, la herramienta Vivado y la tarjeta de desarrollo Nexys4 DDR. Se implementaron distintos ejercicios orientados al uso de lógica combinacional, módulos parametrizables, modulación por ancho de pulso y comunicación serial UART. El trabajo incluyó el diseño de módulos en HDL, su verificación mediante simulación con testbenches, el proceso de síntesis e implementación, y la validación física sobre FPGA. Los resultados obtenidos permitieron comprobar el funcionamiento correcto de los diseños desarrollados y reforzar conceptos fundamentales del diseño digital programable.**

***Palabras clave*—SystemVerilog, FPGA, Vivado, lógica combinacional, multiplexor, PWM, UART, simulación, Nexys4 DDR**

I. Introducción

El diseño de sistemas digitales mediante lenguajes de descripción de hardware (HDL) constituye una práctica fundamental en la ingeniería electrónica moderna. Herramientas como SystemVerilog permiten describir, simular e implementar circuitos digitales de forma estructurada, facilitando el desarrollo de sistemas que posteriormente pueden programarse en dispositivos lógicos reconfigurables como las FPGA.

En este laboratorio se estudió el flujo básico de diseño digital utilizando la herramienta Vivado y la tarjeta de desarrollo Nexys4 DDR basada en una FPGA Artix-7. El proceso de desarrollo incluyó el diseño de módulos en SystemVerilog, la verificación del comportamiento mediante simulación con testbenches, la síntesis del circuito, su implementación en hardware y finalmente la validación física utilizando los periféricos disponibles en la tarjeta de desarrollo.

Durante el desarrollo del laboratorio se implementaron varios ejercicios orientados a comprender distintos aspectos del diseño digital. Entre estos se incluyen el manejo de interruptores y LEDs de la tarjeta, el diseño de módulos combinacionales parametrizables, la generación de señales digitales mediante técnicas de modulación por ancho de pulso y la implementación básica de una interfaz de comunicación serial tipo UART.

II. Marco Teórico

El diseño digital moderno se apoya en la separación del sistema en bloques funcionales, permitiendo organizar la lógica de manera modular. Este enfoque facilita la depuración, la simulación y la posterior implementación del sistema en FPGA [5].

II-A. Diseño modular

El diseño modular consiste en dividir un sistema complejo en varios submódulos más pequeños, cada uno con una función específica. Esta metodología permite reutilizar bloques, verificar componentes de forma independiente y reducir la complejidad del sistema total [5].

II-B. Comunicación serial UART

UART (*Universal Asynchronous Receiver Transmitter*) es un protocolo de comunicación serial asíncrona ampliamente utilizado en sistemas digitales y embebidos. Su operación se basa en el envío secuencial de bits, sin utilizar una señal de reloj compartida entre el transmisor y el receptor [2]. Una trama UART estándar suele incluir:

- un bit de inicio,
- un conjunto de bits de datos,
- opcionalmente un bit de paridad,
- y uno o más bits de parada.

En este laboratorio se trabaja con el formato **8N1**, es decir, 8 bits de datos, sin paridad y 1 bit de parada [1], [2].

II-C. Código ASCII

El código ASCII permite representar caracteres mediante valores numéricos binarios, facilitando su transmisión y procesamiento dentro de sistemas digitales [3]. Gracias a ello, un sistema puede enviar palabras, números y caracteres de control mediante un protocolo serial como UART.

II-D. Sincronización de señales asíncronas

Cuando una señal externa, como un botón, entra al dominio de reloj de un sistema digital, existe riesgo de metastabilidad. Una solución común consiste en usar un sincronizador de dos flip-flops, técnica ampliamente recomendada en diseño digital [4].

II-E. Máquinas de estados finitos

Las máquinas de estados finitos (FSM) se utilizan para controlar secuencias de operación en circuitos digitales. Permiten modelar el comportamiento de un sistema a partir de estados, transiciones y acciones asociadas, siendo una herramienta fundamental en diseños secuenciales [5].

III. Ejercicio 1: Switches, botones y LEDs

III-A. Descripción del ejercicio

El objetivo de este ejercicio consiste en diseñar un módulo de lógica combinacional capaz de calcular el complemento a dos de un número binario de 4 bits. El valor de entrada es introducido mediante interruptores físicos de la tarjeta de desarrollo, mientras que el resultado es visualizado a través de los LEDs disponibles.

Este tipo de ejercicio permite aplicar conceptos fundamentales de diseño digital, como la representación binaria de datos, el uso de lógica combinacional y la interacción entre señales de entrada y salida dentro de una FPGA [6]. Además, constituye una introducción práctica al flujo de desarrollo basado en HDL, simulación funcional, síntesis e implementación sobre hardware programable [9], [8].

III-B. Diseño del módulo

El módulo desarrollado implementa la operación del complemento a dos mediante la siguiente expresión:

$$Y = (\sim X) + 1 \tag{1}$$

donde X corresponde al valor binario ingresado mediante los interruptores y Y representa el resultado mostrado en los LEDs.

El complemento a dos es el método estándar utilizado para representar números enteros con signo en sistemas digitales, debido a que simplifica la implementación de operaciones aritméticas como la resta y el manejo de valores negativos [6]. Desde el punto de vista funcional, la operación se implementó como un bloque puramente combinacional, por lo que la salida depende únicamente del valor instantáneo de la entrada, sin almacenar estados previos.

La descripción del módulo se realizó en SystemVerilog, utilizando una estructura RTL sintetizable. Este enfoque permite modelar de forma clara el comportamiento del circuito y posteriormente traducirlo a hardware mediante las herramientas de síntesis de Vivado [7], [8].

Listing 1: Módulo combinacional `twos_comp4.sv`

```
module twos_comp4 (
    input  logic [3:0] sw,
    output logic [3:0] led
);
    // Complemento a 2: -sw (en 4 bits) == (~sw + 1)
    always_comb begin
        led = (~sw) + 4'd1;
    end
endmodule
```

III-C. Tabla de verdad

El comportamiento esperado del módulo se resume en la Tabla I. En ella se muestran las 16 combinaciones posibles para la entrada de 4 bits y su correspondiente resultado en complemento a dos. Esta tabla fue utilizada como referencia tanto para la simulación como para la validación experimental en la FPGA.

Cuadro I: Tabla de verdad del complemento a dos en 4 bits

B_3	B_2	B_1	B_0	D_3	D_2	D_1	D_0
0	0	0	0	0	0	0	0
0	0	0	1	1	1	1	1
0	0	1	0	1	1	1	0
0	0	1	1	1	1	0	1
0	1	0	0	1	1	0	0
0	1	0	1	1	0	1	1
0	1	1	0	1	0	1	0
0	1	1	1	1	0	0	1
1	0	0	0	1	0	0	0
1	0	0	1	0	1	1	1
1	0	1	0	0	1	1	0
1	0	1	1	0	1	0	1
1	1	0	0	0	1	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	0	1	0
1	1	1	1	0	0	0	1

III-D. Verificación mediante simulación

Para validar el funcionamiento del módulo se desarrolló un banco de pruebas en SystemVerilog. La salida esperada se calculó dentro del propio testbench utilizando la misma definición matemática del complemento a dos. Posteriormente, este valor teórico se comparó con la salida real del módulo. Este tipo de verificación automática permite detectar errores funcionales sin depender únicamente de la inspección visual de las formas de onda, y forma parte de las buenas prácticas de validación en diseño digital [9], [7].

Los resultados de la simulación confirmaron que el módulo produce correctamente el complemento a dos para todas las combinaciones de entrada evaluadas.

Listing 2: Banco de pruebas `tb_twos_comp4.sv`

```
module tb_twos_comp4;
    logic [3:0] sw;
    logic [3:0] led;
    logic [3:0] expected;

    twos_comp4 dut (
        .sw (sw),
        .led(led)
    );

    initial begin
        // Recorre todos los valores posibles de 4
            bits
        sw = 0;
        expected = 0;
        for (int i = 0; i < 16; i++) begin
            sw = $unsigned(i[3:0]);
            expected = (~sw) + 4'd1;
            #1;

            if (led !== expected) begin
                $error("FALLO: sw=%0d (0x%0h) -> led
                    =0x%0h, esperado=0x%0h",
                       sw, sw, led, expected);
                $finish;
            end
        end

        $display("OK: todas las combinaciones
            pasaron.");
        $finish;
    end
endmodule
```

III-E. Implementación en FPGA

Una vez validado el diseño mediante simulación, se procedió a su síntesis e implementación en la FPGA de la tarjeta Nexys4 DDR. Para ello, se utilizó un archivo de restricciones (`XDC`) con el fin de asociar los puertos del módulo superior a los interruptores y LEDs físicos de la tarjeta.

Posteriormente, se generó el bitstream del diseño y se programó la FPGA utilizando el *Hardware Manager* de Vivado. Las pruebas realizadas en la tarjeta confirmaron el comportamiento esperado del sistema: al modificar el valor de los interruptores de entrada, los LEDs mostraban correctamente el complemento a dos correspondiente. Este resultado permitió comprobar la consistencia entre la simulación funcional y la implementación física del diseño [8].

Listing 3: Módulo superior `top.sv` para integración en la Nexys4 DDR

```
module top (
    input  logic [15:0] SW,
    output logic [15:0] LED
);

    // Solo usamos los primeros 4
    twos_comp4 u0 (
        .sw (SW[3:0]),
        .led(LED[3:0])
    );

    // Apagamos los LEDs que no usamos
    assign LED[15:4] = 12'b0;

endmodule
```

III-F. *Verificación mediante simulación*

Para validar el funcionamiento del módulo se desarrolló un banco de pruebas que recorre las 16 combinaciones posibles de entrada, desde 0000 hasta 1111. Los resultados obtenidos confirmaron el correcto funcionamiento del diseño.

IV. Ejercicio 2: Multiplexor 4:1 parametrizable

IV-A. *Introducción*

El diseño digital moderno requiere circuitos reutilizables, escalables y verificables. En este contexto, los multiplexores constituyen uno de los bloques combinacionales más importantes, ya que permiten enrutar información desde varias entradas hacia una sola salida de acuerdo con una señal de control. Su uso es común en sistemas digitales, rutas de datos, unidades aritméticas, procesadores y sistemas programables [6].

En este ejercicio se diseñó un multiplexor $4:1$ parametrizable en SystemVerilog. La característica principal del diseño consiste en que el ancho del bus de datos no está fijado de manera rígida, sino que puede ajustarse mediante un parámetro. Esto permite reutilizar el mismo módulo para palabras de diferentes tamaños, como 4, 8 o 16 bits, sin necesidad de reescribir el circuito.

Además del diseño del módulo, se implementó un banco de pruebas auto-verificable (*self-checking testbench*) que permitió validar el comportamiento del multiplexor para tres configuraciones de ancho de bus: 4, 8 y 16 bits. Finalmente, se llevó a cabo la síntesis del diseño en Vivado, con el objetivo de verificar que el circuito fuera completamente combinacional y no infiriera memoria no deseada, especialmente *latches*.

IV-B. *Objetivos*

Los objetivos de este ejercicio fueron los siguientes:

- Diseñar un multiplexor $4:1$ parametrizable utilizando SystemVerilog.
- Implementar el diseño bajo una metodología RTL clara, sintetizable y libre de ambigüedades.
- Utilizar buenas prácticas de modelado, específicamente el uso de `logic` y `always_comb`.
- Verificar el funcionamiento del diseño mediante un testbench que pruebe distintas configuraciones del parámetro `WIDTH`.
- Analizar el resultado de la síntesis para comprobar que el circuito es completamente combinacional y que no se infieren elementos secuenciales.

IV-C. *Fundamento teórico*

IV-C1. Concepto de multiplexor: Un multiplexor es un circuito combinacional encargado de seleccionar una de varias entradas de datos y transferirla a una única salida. El número de líneas de selección depende de la cantidad de entradas. Para un multiplexor $4:1$, se requieren dos bits de selección porque:

$$2^2 = 4$$

Esto significa que las combinaciones posibles del selector permiten escoger exactamente una entre cuatro entradas.

El comportamiento funcional del multiplexor $4:1$ puede expresarse mediante la siguiente ecuación por casos:

$$y = \begin{cases} d_0 & \text{si } sel = 00 \\ d_1 & \text{si } sel = 01 \\ d_2 & \text{si } sel = 10 \\ d_3 & \text{si } sel = 11 \end{cases}$$

Desde el punto de vista lógico, el multiplexor no almacena información previa, por lo que se clasifica como un circuito puramente combinacional [6].

IV-C2. Parametrización del ancho de palabra: En lugar de diseñar un multiplexor exclusivamente de 1 bit o de 8 bits, en este ejercicio se implementó un módulo generalizado mediante un parámetro llamado `WIDTH`. Este parámetro representa el número de bits de cada entrada y de la salida.

La parametrización es una característica esencial en el diseño de hardware reutilizable, ya que permite adaptar el mismo bloque funcional a distintos tamaños de bus sin alterar la estructura del módulo [7].

IV-C3. Uso de `logic` en SystemVerilog: En Verilog tradicional existe una distinción entre tipos como `wire` y `reg`. Sin embargo, SystemVerilog introduce el tipo `logic`, el cual simplifica la descripción RTL al poder emplearse como tipo unificado para la mayoría de señales internas y de interfaz [7].

En este ejercicio se decidió utilizar exclusivamente `logic`, siguiendo la recomendación de modelar el circuito con sintaxis moderna y consistente.

IV-C4. Uso de `always_comb`: Para describir lógica combinacional se utilizó el bloque `always_comb`. Esta construcción tiene la ventaja de expresar explícitamente la intención de describir lógica sin memoria, a diferencia de bloques más generales como `always @(*)`. Además, `always_comb` ayuda a prevenir errores derivados de listas de sensibilidad incompletas y favorece una síntesis más robusta [7], [8].

IV-D. *Especificación funcional del diseño*

El módulo implementado posee la siguiente estructura general:

- Cuatro entradas de datos: `d0`, `d1`, `d2`, `d3`
- Una señal selectora de dos bits: `sel[1:0]`
- Una salida de datos: `y`
- Un parámetro: `WIDTH`

La salida corresponde exactamente a una de las entradas, elegida de acuerdo con el valor del selector.

IV-E. *Tabla de verdad*

El comportamiento funcional del multiplexor $4:1$ se describe en el Cuadro II. En él se muestra cómo la señal de selección determina cuál de las cuatro entradas de datos es transmitida hacia la salida del circuito.

Cuadro II: Tabla de verdad del multiplexor 4:1

sel[1:0]	Salida
00	d_0
01	d_1
10	d_2
11	d_3

Cada fila del cuadro representa una posible combinación de la señal selectora. Cuando el selector toma el valor `00`, la salida del multiplexor corresponde a la entrada d_0. De manera similar, cuando el selector toma los valores `01`, `10` o `11`, la salida corresponde respectivamente a las entradas d_1, d_2 y d_3.

Es importante notar que esta tabla de verdad describe el comportamiento lógico del multiplexor independientemente del ancho del bus de datos. En el diseño implementado, cada entrada corresponde a un bus de `WIDTH` bits, por lo que la selección se aplica de manera paralela a todos los bits del bus.

IV-F. *Diseño del módulo RTL*

El diseño del multiplexor se realizó en SystemVerilog mediante una estructura simple, clara y sintetizable. A continuación se analiza el código por fragmentos.

IV-F1. Definición del módulo y parámetro: Un primer fragmento importante es la definición del parámetro de ancho:

```
module mux4to1 #(
    parameter int WIDTH = 8
)(
```

Este fragmento indica que el módulo se llama `mux4to1` y que posee un parámetro entero llamado `WIDTH`. El valor por defecto es 8, lo que significa que, si no se especifica otra configuración al instanciarlo, el módulo trabajará con buses de 8 bits.

La razón de esta decisión de diseño es lograr un módulo flexible. En lugar de escribir versiones separadas del multiplexor para 4, 8 y 16 bits, se construyó un único bloque parametrizable.

IV-F2. Declaración de puertos: Los puertos del módulo se declararon así:

```
input  logic [WIDTH-1:0] d0,
input  logic [WIDTH-1:0] d1,
input  logic [WIDTH-1:0] d2,
input  logic [WIDTH-1:0] d3,
input  logic [1:0]       sel,
output logic [WIDTH-1:0] y
```

Aquí se observa que cada entrada de datos tiene exactamente `WIDTH` bits, mientras que la señal `sel` se mantiene de 2 bits porque el número de entradas del multiplexor es fijo. La salida también tiene `WIDTH` bits.

Esto garantiza consistencia de tamaños y permite que la selección de la entrada ocurra sobre palabras completas.

IV-F3. Lógica combinacional: La parte central del diseño es:

```
always_comb begin
    case (sel)
        2'b00:   y = d0;
        2'b01:   y = d1;
        2'b10:   y = d2;
        2'b11:   y = d3;
        default: y = '0;
    endcase
end
```

Este bloque implementa directamente la tabla de verdad del multiplexor. La instrucción `case` evalúa el valor del selector y asigna a la salida una de las cuatro entradas.

La inclusión del caso `default` es muy importante. Aunque idealmente el selector solo debe tomar los valores 00, 01, 10 y 11, en simulación pueden aparecer estados indefinidos como `X` o `Z`. Si no existiera una asignación por defecto, el sintetizador podría interpretar que la salida debe conservar un valor previo en algunos casos, lo que llevaría a inferir memoria no deseada. Por esta razón, la asignación `default: y = '0;` se incluyó como una medida de robustez y para garantizar un diseño libre de latches [8].

IV-G. *Explicación del testbench*

La verificación del circuito se realizó mediante un banco de pruebas auto-comprobable. A diferencia de un testbench interactivo, en este caso no es necesario introducir manualmente estímulos, ya que el propio banco de pruebas genera los vectores de entrada, calcula el valor esperado y determina si la salida del DUT es correcta.

IV-G1. Estrategia general de verificación: La estrategia seguida fue la siguiente:

1. Instanciar tres versiones del multiplexor, una con `WIDTH=4`, otra con `WIDTH=8` y otra con `WIDTH=16`.
2. Generar automáticamente 50 conjuntos de valores pseudoaleatorios para las entradas de cada caso.
3. Aplicar las cuatro combinaciones del selector en cada conjunto de datos.
4. Comparar la salida del DUT con el valor esperado calculado en el propio testbench.
5. Mostrar mensajes de `PASS` o `ERROR` según corresponda.

IV-G2. Instanciación de distintos anchos: Uno de los objetivos principales del ejercicio era comprobar que el parámetro funcionara correctamente. Por ello, el testbench no se limitó a una sola instancia del módulo, sino que generó tres configuraciones distintas.

Por ejemplo, para el caso de 4 bits se definió:

```
localparam int W4 = 4;
logic [W4-1:0] d0_4, d1_4, d2_4, d3_4, y_4;
logic [1:0] sel_4;
mux4to1 #(.WIDTH(W4)) dut4 ( ... );
```

La misma idea se repitió para 8 y 16 bits. Esta decisión evitó problemas de compatibilidad con rangos variables en algunos analizadores y, además, hizo más explícita la verificación de cada configuración.

IV-G3. Generación de estímulos: Dentro de cada tarea de prueba se generaron datos pseudoaleatorios con `$urandom`. Por ejemplo:

```
d0_8 = $urandom;
d1_8 = $urandom;
d2_8 = $urandom;
d3_8 = $urandom;
```

Aunque `$urandom` produce valores de mayor ancho, estos son truncados automáticamente al tamaño del bus correspondiente. Esta técnica permite probar el diseño con múltiples combinaciones de entrada sin necesidad de enumerarlas manualmente.

IV-G4. Prueba de todas las selecciones: Para cada conjunto de entradas se probaron las cuatro combinaciones del selector mediante un ciclo:

```
for (int s = 0; s < 4; s++) begin
    sel_8 = s;
    #1;
    ...
end
```

De esta forma, en cada iteración se verificó el comportamiento del multiplexor para `sel = 00`, `01`, `10` y `11`. La espera de `#1` permite que la salida combinacional se estabilice antes de compararla con el valor esperado.

IV-G5. Cálculo del valor esperado: El valor esperado se calculó dentro del testbench usando la misma lógica funcional del multiplexor:

```
case (sel_8)
    2'b00: expected = d0_8;
    2'b01: expected = d1_8;
    2'b10: expected = d2_8;
    2'b11: expected = d3_8;
    default: expected = '0;
endcase
```

Esto permite comparar la salida real del DUT con la salida que teóricamente debería producirse.

IV-G6. Auto-verificación: La comparación se realizó con:

```
if (y_8 !== expected) begin
    errors++;
    $display("ERROR␣...");
end
```

El uso de `!==` en lugar de `!=` es importante porque permite detectar también estados indeterminados. Si la salida fuera incorrecta o estuviera en un estado no definido, el testbench lo reportaría.

Finalmente, si no se detectaban errores, se imprimía un mensaje `PASS`. Si se detectaba al menos un fallo, la simulación se detenía mediante `$fatal`. Este enfoque convierte al banco de pruebas en una herramienta de verificación automática, reduciendo la dependencia del análisis manual de formas de onda [9].

IV-H. Proceso de simulación

La simulación se ejecutó en Vivado utilizando el simulador XSim. El procedimiento general fue el siguiente:

1. Crear un proyecto RTL.
2. Agregar el archivo `mux4to1.sv` como fuente de diseño.
3. Agregar `tb_mux4to1.sv` como fuente de simulación.
4. Establecer el testbench como top de simulación.
5. Ejecutar la simulación conductual (*Run Behavioral Simulation*).

Durante la simulación se obtuvo la siguiente secuencia de resultados:

- PASS WIDTH=4 (checks=200)
- PASS WIDTH=8 (checks=200)
- PASS WIDTH=16 (checks=200)

Esto indica que:

$$50 \text{ conjuntos de datos} \times 4 \text{ selecciones} = 200 \text{ pruebas por ancho}$$

y, en total:

$$200 \times 3 = 600 \text{ pruebas}$$

sin errores.

IV-I. Resultados de simulación

La salida de la consola del simulador confirmó que todas las pruebas fueron superadas satisfactoriamente. Esto demuestra que la implementación del multiplexor coincide con su especificación funcional para los tres anchos evaluados.

La simulación cumplió con lo solicitado por el enunciado, tanto en cantidad de datos de prueba como en cantidad de configuraciones evaluadas.

IV-J. Proceso de síntesis

Una vez validado el diseño mediante simulación, se ejecutó la síntesis en Vivado para verificar que el circuito fuera efectivamente sintetizable y completamente combinacional.

El proceso seguido fue:

1. Ejecutar *Run Synthesis*.
2. Abrir el diseño sintetizado.
3. Revisar el esquemático generado.
4. Analizar el reporte de utilización.
5. Revisar el resumen de temporización.

IV-K. *Análisis del esquemático sintetizado*

El esquemático sintetizado mostró que el circuito se implementó mediante LUT6 e I/O buffers. No se observaron flip-flops ni latches, lo cual confirma que la descripción RTL fue interpretada como lógica combinacional pura.

En particular, para el caso de 8 bits, la síntesis generó 8 LUTs. Esto se explica porque cada bit de la salida requiere una lógica equivalente a un multiplexor 4:1 de un bit. En consecuencia, el multiplexor completo de 8 bits puede entenderse como ocho multiplexores de 1 bit operando en paralelo.

IV-L. *Reporte de utilización*

El reporte de utilización indicó:

- Slice LUTs: 8
- Bonded IOB: 42

El valor de 42 IOBs coincide exactamente con el número de entradas y salidas del módulo sintetizado:

$$8 + 8 + 8 + 8 + 2 + 8 = 42$$

Esto corresponde a las cuatro entradas de 8 bits, el selector de 2 bits y la salida de 8 bits.

Además, el hecho de que no se reportaran flip-flops ni latches evidencia que no existe almacenamiento secuencial en el diseño.

IV-M. *Reporte de temporización*

En el resumen de temporización se indicó que no existían restricciones de temporización especificadas por el usuario. Esto es normal porque el diseño no contiene reloj. Al ser un circuito puramente combinacional, no existen caminos síncronos entre registros que requieran análisis de *setup* o *hold* como en circuitos secuenciales [9], [8].

Por tanto, el resultado del timing no representa un problema, sino una consecuencia natural de la arquitectura del circuito.

IV-N. *Discusión de diseño*

Desde el punto de vista del diseño, la implementación resultó adecuada por varias razones. En primer lugar, la parametrización permitió un modelo reutilizable y escalable. En segundo lugar, el uso de `always_comb`, `logic` y `default` favoreció una descripción clara, consistente y segura para síntesis. Finalmente, el testbench auto-comprobable permitió una validación robusta, eliminando la necesidad de una inspección manual extensa de formas de onda.

En conjunto, el ejercicio permitió recorrer un flujo completo de diseño digital: especificación, implementación RTL, verificación funcional, síntesis y análisis de resultados.

IV-Ñ. *Conclusiones*

Se implementó correctamente un multiplexor $4:1$ parametrizable en SystemVerilog. El uso del parámetro `WIDTH` permitió reutilizar el mismo módulo para buses de distintos anchos, y el banco de pruebas validó su funcionamiento para 4, 8 y 16 bits.

La simulación confirmó el correcto funcionamiento del diseño mediante 600 verificaciones automáticas. Posteriormente, la síntesis mostró que el circuito fue implementado usando únicamente lógica combinacional, sin inferencia de latches ni flip-flops.

En conclusión, el ejercicio permitió consolidar conceptos fundamentales de diseño combinacional, parametrización, verificación automática y análisis posterior a la síntesis, todos ellos esenciales en el desarrollo de sistemas digitales programables.

V. Ejercicio 3: Diseño del módulo PWM

V-A. *Introducción*

La modulación por ancho de pulso (PWM) es una técnica que mantiene la frecuencia constante mientras varía el ciclo de trabajo, controlando así la potencia promedio entregada a una carga [10]. En este ejercicio se implementa un módulo PWM de 4 bits sobre la tarjeta Nexys 4 DDR con FPGA Artix-7 y reloj de 100 MHz [12].

V-B. *Diseño del módulo*

Con un reloj de 100 MHz, el período de 1 ms requiere:

$$N_{total} = \frac{T_{PWM}}{T_{clk}} = \frac{1 \times 10^{-3}}{10 \times 10^{-9}} = 100{,}000 \text{ ciclos} \quad (2)$$

Dividiendo entre los 16 niveles del código de 4 bits:

$$N_{nivel} = \frac{100{,}000}{16} = 6{,}250 \text{ ciclos por nivel} \quad (3)$$

El ciclo de trabajo del nivel k es $D_k = \frac{k}{16} \times 100\,\%$. La señal `pwm_out` se activa cuando el contador de niveles es menor al valor de `duty`, generando directamente el ciclo de trabajo deseado [8].

Listing 4: Módulo pwm_4bit.sv

```
module pwm_4bit (
    input  logic        clk,
    input  logic        rst,
    input  logic [3:0]  duty,
    output logic        pwm_out
);
    integer      counter;
    logic [3:0]  level_counter;

    always_ff @(posedge clk or posedge rst) begin
        if (rst) begin
            counter       <= 0;
            level_counter <= 0;
        end else begin
            if (counter >= 6249) begin
                counter       <= 0;
                level_counter <= level_counter + 1;
            end else
                counter <= counter + 1;
        end
    end

    assign pwm_out = (level_counter < duty) ? 1'b1 :
         1'b0;
endmodule
```

V-C. *Testbench exhaustivo*

El testbench verifica el módulo mediante cinco pruebas independientes [13]. Para cada valor de `duty`, aplica reset, mide exactamente 100,000 ciclos y compara el ciclo de trabajo medido con el esperado (tolerancia $\pm 1\,\%$):

$$D_{medido} = \frac{N_{HIGH}}{N_{total}} \times 100\,\% \tag{4}$$

Cuadro III: Resultados del testbench exhaustivo

Test	Descripción	Resultado
1	Reset asíncrono: `pwm_out`= 0 durante `rst`	PASS
2	Barrido completo `duty` 0–15 (16 valores)	16/16 PASS
3a	Caso límite `duty`= 0: salida siempre LOW	PASS
3b	Caso límite `duty`= 15: $N_{HIGH} = 93{,}750$	PASS
4	Cambio dinámico (4→12) sin reset	PASS
5	Período medido: 1,000,000 ns = 1 ms	PASS

Todos los casos obtuvieron PASS con error de medición de 0

V-D. *Validación en tarjeta*

El diseño fue sintetizado sobre la Nexys 4 DDR (xc7a100tcsg324-1) con Vivado. Al incrementar los interruptores de `0000` a `1111` se observaron 16 niveles de brillo distintos en el LED, confirmando el correcto funcionamiento del módulo PWM en hardware.

V-E. *Conclusión del ejercicio*

Se implementó exitosamente un módulo PWM de 4 bits con período de 1 ms. El testbench exhaustivo verificó los 16 niveles posibles con error nulo, validando tanto el reset asíncrono como el comportamiento dinámico del diseño.

VI. Ejercicio 4: Implementación del Sistema UART

Este ejercicio consiste en el diseño de un sistema de comunicación serial basado en UART (*Universal Asynchronous Receiver Transmitter*), utilizado para transmitir datos de forma asíncrona entre dispositivos [1], [2].

El sistema, implementado en *SystemVerilog*, transmite el mensaje `"Hola mundo"` seguido de `CR` y `LF` al presionar un botón, y muestra en LEDs cualquier dato recibido.

VII. Descripción General del Sistema

El sistema utiliza como entradas el reloj, reset, botón de envío y la señal UART de entrada. Como salidas, genera la señal de transmisión UART y un conjunto de LEDs.

VII-A. *Objetivo*

- transmitir un mensaje mediante UART;
- recibir datos y mostrarlos en LEDs;
- controlar el sistema mediante una FSM;
- validar el funcionamiento mediante simulación.

VIII. Arquitectura del Sistema

Cuadro IV: Bloques principales del sistema

Bloque	**Entrada**	**Función**
Baud rate	clk	genera `tick_16x`
UART RX	uart_rx	recibe datos
UART TX	tx_data	transmite datos
FSM	señales de control	controla envío
LEDs	rx_data	muestra datos

Cada bloque cumple una función específica, permitiendo una arquitectura modular y fácil de verificar.

IX. Representación ASCII

Los datos se transmiten en formato ASCII, donde cada carácter se representa con 8 bits [3].

Cuadro V: Ejemplo ASCII del mensaje

Carácter	Decimal	Hex	Binario
H	72	48h	01001000
o	111	6Fh	01101111
espacio	32	20h	00100000
CR	13	0Dh	00001101
LF	10	0Ah	00001010

Los LEDs muestran directamente estos valores binarios cuando se recibe un dato válido.

X. Máquina de Estados Finita

Cuadro VI: Estados de la FSM

Estado	Código	Función
S_IDLE	00	espera botón
S_SEND	01	envía dato
S_WAIT	10	espera transmisión

Cuadro VII: Transiciones principales

Estado	Condición	Siguiente
IDLE	btn_rise	SEND
SEND	tx_ready	WAIT
WAIT	tx_busy=0	SEND / IDLE

La FSM permite enviar el mensaje carácter por carácter de forma ordenada.

XI. Lógica del Sistema

XI-A. *Detector de flanco*

```
btn_rise = btn_sync & ~btn_sync_d
```
(5)

Cuadro VIII: Detector de flanco

btn_sync	btn_sync_d	btn_rise
0	0	0
1	0	1
1	1	0

XI-B. *Registro de LEDs*

Cuadro IX: Comportamiento de LEDs

rst_n	rx_valid	Acción
0	X	limpia LEDs
1	0	mantiene valor
1	1	carga dato

XI-C. *Control de transmisión*

Cuadro X: Condición de transmisión

Estado	tx_ready	tx_start
SEND	1	1
otro	X	0

XII. Señales principales

Cuadro XI: Señales del sistema

Señal	Función
clk	reloj
rst_n	reset
uart_rx	entrada serial
uart_tx	salida serial
leds	visualización
tx_data	dato a enviar
rx_data	dato recibido
state	estado FSM

XIII. Verificación mediante Testbench

La simulación permite validar el sistema antes de implementarlo [5].

Se verifica que:

- el mensaje se transmite correctamente;
- los datos se envían en orden;
- los LEDs reflejan los datos recibidos;
- la FSM retorna a reposo.

XIV. Uso de Inteligencia Artificial como apoyo

Se utilizó inteligencia artificial para mejorar la redacción, organización y claridad del documento. [13]

El diseño, implementación y pruebas del sistema fueron realizados por el autor.

Cuadro XII: Uso de IA en el proyecto

Actividad	Uso de IA
Redacción	mejora de texto técnico
Estructura	organización del documento
Explicaciones	apoyo conceptual

XV. Conclusiones

El módulo `uart_top` integra correctamente los bloques fundamentales de un sistema UART, incluyendo temporización, recepción, transmisión y control secuencial. Esta integración permite implementar una interfaz serial funcional, capaz de comunicarse con un equipo externo y mostrar localmente la información recibida [1], [2].

La arquitectura modular empleada facilita la organización del diseño, mejora la legibilidad del código y simplifica el proceso de verificación, de acuerdo con prácticas comunes en flujos de desarrollo FPGA [5]. Asimismo, el uso de un sincronizador de dos flip-flops para el botón constituye una solución adecuada para tratar señales asíncronas y reducir riesgos de metastabilidad [4].

Las tablas incluidas en este documento permiten describir formalmente el comportamiento del sistema. En particular, las tablas del detector de flanco, del registro de LEDs, de la FSM y del mensaje transmitido establecen una relación clara entre el código fuente y la operación esperada del hardware.

XV-A. *Repositorio del Proyecto*

El código fuente completo del sistema está disponible en GitHub en el repositorio:
`github.com/WallyTEC/Taller-Diseno-Digital`

Referencias

[1] Microchip Technology Inc., *Section 21. UART*, PIC32 Family Reference Manual. Disponible en: https://ww1.microchip.com/downloads/en/DeviceDoc/61107E.pdf

[2] Microchip Technology Inc., *TB3208: Basic Operation of UART with Protocol Support*. Disponible en: https://ww1.microchip.com/downloads/aemDocuments/documents/OTH/ApplicationNotes/ApplicationNotes/TB3208-Basic-Operation-of-UART-with-Protocol-Support-90003208A.pdf

[3] National Institute of Standards and Technology (NIST), *American National Standard Code for Information Interchange*, FIPS PUB 1-2. Disponible en: https://nvlpubs.nist.gov/nistpubs/Legacy/FIPS/fipspub1-2-1977.pdf

[4] Intel Corporation, *AN 545: Design Guidelines and Timing Closure Techniques for HardCopy ASICs*. Disponible en: https://cdrdv2-public.intel.com/654507/an545.pdf

[5] Intel Corporation, *Intel Quartus Prime Standard Edition User Guides*. Disponible en: https://www.intel.com/programmable/technical-pdfs/qps-ugs.pdf

[6] M. M. Mano y M. D. Ciletti, *Digital Design*, 6th ed., Pearson, 2017.

[7] IEEE, *IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2018.

[8] AMD Xilinx, *Vivado Design Suite User Guide: Synthesis (UG901)*, 2023.

[9] AMD Xilinx, *Vivado Design Suite User Guide: Logic Simulation (UG900)*, 2023.

[10] M. H. Rashid, *Power Electronics Handbook*, 3rd ed. Burlington, MA: Butterworth-Heinemann, 2011.

[11] D. W. Hart, *Power Electronics*. New York, NY: McGraw-Hill, 2010.

[12] Digilent Inc., *Nexys 4 DDR FPGA Board Reference Manual*, 2016. [Online]. Available: https://digilent.com/reference/programmable-logic/nexys-4-ddr/reference-manual

[13] C. Spear y G. Tumbush, *SystemVerilog for Verification*, 3rd ed. New York, NY: Springer, 2012.

- `https://chatgpt.com/share/69bc5046-8db0-800d-a86d-2db88af211d3`
- `https://chatgpt.com/share/69bc5007-a478-800d-8572-8dfa962befa7`
- `https://chatgpt.com/share/69bc53ee-dc0c-800d-a06f-c71aaf97305f`